

FORMATION PYTHON

J3

Fichiers, formats et robustesse

pathlib, CSV et JSON réels, hiérarchie des exceptions, erreurs métier personnalisées — et l'organisation du code en paquet.

05-fichiers-et-formats

06-erreurs-et-exceptions

07-modules-et-organisation

Objectifs de la journée



CSV et JSON en production

DictReader/Writer, encodages, newline, json.load/dump et leurs options.



Une gestion d'erreurs complète

Hierarchie des exceptions, else/finally, raise, exceptions métier.



Organiser en paquet

Le paquet fiscalite/ : __init__.py, imports, une responsabilité par module.



Maîtriser uv

pyproject.toml, uv.lock, uv add/sync/run — d'où viennent les bibliothèques.

LA SEMAINE

J1

Fondamentaux du langage

J2

Fonctions & structures

J3

Fichiers & robustesse

J4

Objets & pandas

J5

Intégration & projets

Fil rouge : normaliser les déclarations.

pathlib : les chemins comme des objets



L'opérateur / assemble

DATA / "sortie" / "regions.csv" — portable Windows et Linux, sans collage de chaînes.



Relatif au script, pas au terminal

Path(__file__).parent : le script marche d'où qu'on le lance.



exists, mkdir, stem, suffix

Tester, créer (parents=True, exist_ok=True), décomposer un nom.



glob pour énumérer

DATA.glob("*.csv") : tous les fichiers d'un motif — la base des traitements par lot.



pathlib

```
from pathlib import Path

DATA = Path(__file__).parent \
    .parent / "00-data" / "sortie"

DATA.exists()          # True
SORTIE.mkdir(parents=True,
              exist_ok=True)

for f in sorted(DATA.glob("*.csv")):
    print(f.stem, f.stat().st_size)
contribuables 2148000
```

Ouvrir : with, encodage, modes



with ferme TOUJOURS

Même si une exception éclate au milieu — plus de fichiers zombies ni de données perdues.



encoding="utf-8", non négociable

Sans lui, Windows lit en cp1252 : « Bafatá » devient « BafatÃ¡ » selon le poste.



Les modes : r, w, a

w écrase, a ajoute — un rapport journalier s'ouvre en a, jamais en w.



Deux fichiers dans un with

with open(...) as fin, open(...) as fout: — lecture et écriture appariées.



with & encodage

```
with open(DATA / "regions.csv",
          encoding="utf-8") as f:
    entete = f.readline()
# fermé ici, même en cas d'erreur

# lecture + écriture appariées
with open(src, encoding="utf-8") as fin, \
    open(dst, "w", newline="",
          encoding="utf-8") as fout:
    ...
```

CSV en lecture : DictReader et ses pièges



Chaque ligne devient un dict

La 1re ligne fournit les clés : `ligne["nif"]`, `ligne["code_region"]`.



TOUT arrive en str

"45000000", "" ou "abc" — la conversion est votre responsabilité (module 06).



Champ vide ≠ champ absent

`ligne["telephone"]` vaut "" : tester avec `if not valeur`, pas avec `in`.



delimiter si besoin

`DictReader(f, delimiter=";")` pour les exports Excel français.



DictReader

```
import csv

with open(DATA / "contribuables.csv",
          encoding="utf-8") as f:
    for ligne in csv.DictReader(f):
        code = ligne["code_region"]
        compte[code] = \
            compte.get(code, 0) + 1

Bissau : 2513      Gabú : 2461 ...

# export Excel FR : delimiter=";"
```

CSV en écriture : DictWriter, proprement



fieldnames fixe les colonnes

Reprendre lecteur.fieldnames garantit le même schéma que la source.



writeheader() d'abord

Sans lui, le fichier produit n'a pas d'en-tête — illisible en aval.



newline="" en écriture

Sinon Windows double les sauts de ligne — LE bug CSV le plus fréquent.



Filtrer en streamant

Lire, tester, écrire ligne à ligne : aucun fichier entier en mémoire.

```
                                extrait_bissau.csv
with open(src, encoding="utf-8") as fin, \
    open(dst, "w", newline="",
         encoding="utf-8") as fout:
    lecteur = csv.DictReader(fin)
    scribe = csv.DictWriter(fout,
                           fieldnames=lecteur.fieldnames)
    scribe.writeheader()
    for ligne in lecteur:
        if ligne["code_region"] == "BIS":
            scribe.writerow(ligne)
```

JSON : structures imbriquées et options



json.load → dicts et listes

Le fichier devient exactement les structures du J2 — mêmes réflexes.



Naviguer par clés successives

`d["identite"]["ville"], d["etablissements"][0]["actif"]`.



any() / all() sur l'imbriqué

« au moins un établissement actif » tient en une expression.



dump : ensure_ascii=False

Sinon « Gabú » sort en `\u00fa` ; `indent=2` pour un fichier lisible.



dossiers imbriqués

```
import json

with open(DATA / "dossiers_fiscaux.json",
          encoding="utf-8") as f:
    dossiers = json.load(f)

n = sum(1 for d in dossiers
        if any(e["actif"]
               for e in d["etablissements"]))

json.dump(res, f, ensure_ascii=False,
          indent=2)
```

Lire un traceback, connaître la hiérarchie



Lire de bas en haut

Dernière ligne : QUOI (le type + le message). Remontée : OÙ (la pile d'appels).



Les exceptions sont des classes

ValueError, KeyError, FileNotFoundError... héritent toutes d'Exception.



Attraper un parent attrape les enfants

except OSError attrape FileNotFoundError ET PermissionError.



except: nu = interdit

Il avale même Ctrl-C et les fautes de frappe — attraper PRÉCIS, toujours.



anatomie d'une erreur

```
Traceback (most recent call last):  
  File "tp.py", line 12, in <module>  
    m = lire_montant(ligne["ca"])  
  File "tp.py", line 5, in lire_montant  
    return int(txt)  
ValueError: invalid literal for int()
```

hiérarchie (extrait)

Exception

```
├─ ValueError      int("abc")  
├─ KeyError        d["absent"]  
└─ OSError — FileNotFoundError
```


try / except / else / finally : les 4 temps



try : le strict minimum

Une seule opération risquée par try — sinon on ne sait plus QUI a échoué.



except cible et récupère

Compter, journaliser, valeur de repli — mais jamais avaler en silence.



else : le chemin du succès

S'exécute si AUCUNE exception — sépare le risqué du reste.



finally : quoi qu'il arrive

Nettoyage garanti — c'est exactement ce que with automatise.



les 4 clauses

```
try:
    montant = int(ligne["ca"])
except ValueError:
    invalides += 1
except KeyError:
    colonnes_absentes += 1
else:
    # succès seulement
    total += montant
finally:
    # toujours
    lues += 1
```

raise et VOS exceptions métier



raise applique vos règles

Un montant vide ou négatif n'est pas une erreur Python — c'est VOTRE règle : levez-la.



Une classe d'exception métier

`class MontantInvalide(ValueError)` — trois lignes, et l'appelant cible précisément.



Le message porte le contexte

`raise MontantInvalide(f"NIF {nif} : {txt!r}")` — le débogage est déjà fait.



from enchaîne les causes

`raise MontantInvalide(...) from exc` garde le traceback d'origine.



exception métier

```
class MontantInvalide(ValueError):  
    """Montant vide ou négatif."""  
  
def lire_montant(txt: str) -> int:  
    if not txt.strip():  
        raise MontantInvalide("vide")  
    m = int(txt) # ValueError si non num.  
    if m < 0:  
        raise MontantInvalide(f"{m}")  
    return m  
  
except MontantInvalide: rejets += 1
```

Quatre formats de date dans UN fichier

2025-03-14

14/03/2025

14-3-2025

20250314

date_depot dans declarations.csv — exactement comme dans les fichiers administratifs réels



Essayer chaque format en cascade

strptime dans un try/except ValueError, format après format, continue.



Renvoyer None si tout échoue

L'appelant décide : écarter, corriger, signaler — la fonction ne tranche pas.



La liste FORMATS est une donnée

Un 5e format apparaît ? Une ligne à ajouter, zéro logique à toucher.



le normalisateur

```
FORMATS = ["%Y-%m-%d", "%d/%m/%Y",
            "%d-%m-%Y", "%Y%m%d"]

def normaliser(brute: str):
    for fmt in FORMATS:
        try:
            d = strptime(brute.strip(), fmt)
            return d.strftime("%Y-%m-%d")
        except ValueError:
            continue
    return None
```

Du fichier unique au paquet réutilisable

```
07-modules-et-organisation/
├── fiscalite/           # le paquet
│   ├── __init__.py     # l'interface
│   ├── dates.py        # normaliser()
│   └── penalites.py    # penalite()
└── tp_utiliser_paquet.py

# __init__.py expose l'essentiel :
from .dates import normaliser
from .penalites import penalite
```

```
from fiscalite import normaliser, penalite
penalite(8_100_000, 45) # → 972000
```



Un module = une responsabilité

dates.py ne parle que de dates ; penalites.py que de pénalités — testables séparément.



__init__.py définit l'interface

from fiscalite import penalite — l'utilisateur ignore le découpage interne.



Le point du relatif

from .dates import ... : le point dit « dans CE paquet », pas ailleurs.



if __name__ == "__main__"

Un module s'importe ET se lance : ce garde évite l'exécution à l'import.

uv : d'où viennent les bibliothèques



pyproject.toml = la demande

pandas>=2.2 : ce dont le projet a besoin, en termes lâches.



uv.lock = la réponse exacte

pandas 2.2.3 précisément — versionné, donc identique pour tous.



uv add écrit, uv sync applique

uv add openpyxl met à jour les deux fichiers ; jamais d'édition à la main.



uv run : le bon interpréteur

uv run python tp.py utilise .venv sans l'activer — zéro cérémonie.



uv

```
# pyproject.toml (extrait)
[project]
requires-python = ">=3.12"
dependencies = [
    "pandas>=2.2", "jupyterlab>=4.2",
]

$ uv add openpyxl    # ajoute + verrouille
$ uv sync            # reproduit .venv
$ uv run python tp_lecture.py

# uv.lock : versions exactes, comité
```

Les vraies données entrent en scène

- 1 **tp_lecture.py** — référentiel, comptage par région, extrait Bissau, JSON
- 2 **tp_dates_sales.py** — normaliser 4 formats + lire_montant avec raise
- 3 **fiscalite/penalites.py** — transférer vos fonctions du J2 dans le paquet
- 4 **tp_utiliser_paquet.py** — importer et utiliser votre propre paquet
- 5 **Pour les rapides** — une exception MontantInvalide dédiée + glob sur sortie/



Fichiers du jour

```
05-fichiers-et-formats/  
06-erreurs-et-exceptions/  
07-modules-et-organisation/  
    fiscalite/
```



Régénérez les données si besoin : `cd 00-data
&& python generate_data.py --auto`

À retenir

- ✓ **pathlib + with + utf-8** — le trio d'ouverture — et glob pour traiter des lots de fichiers.
- ✓ **DictReader / DictWriter** — tout arrive en str, `newline=""` en écriture : les deux pièges sont connus.
- ✓ **except PRÉCIS** — la hiérarchie est comprise ; `else/finally` séparent succès et nettoyage.
- ✓ **Exceptions métier** — `MontantInvalide` : vos règles ont leurs propres erreurs, ciblables.
- ✓ **Le paquet fiscalite** — votre code du J2, devenu importable dans tout le dépôt.



Demain — J4 : objets et pandas

dataclasses complètes (`field`, `frozen`, `__post_init__`), `loc/iloc`, dates pandas, `groupby` et `merge` en détail. Et : lancement du projet final.